

B.TECH 1ST YEAR · COMPUTER SCIENCE & ENGINEERING

Data Structures

Unit II — Linked Lists

Comprehensive Study Material

Singly Linked List

Doubly Linked List

Circular Linked List

Dynamic Memory

Operations & Algorithms

Arrays vs Linked Lists

Applications

Exam Q&A

Prepared for **University Examinations** · Covers Complete Syllabus
Includes Diagrams · Algorithms · Time Complexity · Q&A · Quick Revision



Table of Contents

Section 1 — Introduction to Linked Lists	Page 3
Definition, Need, Terminology	
Section 2 — Need for Linked Lists	Page 4
Dynamic Memory Allocation, Limitations of Arrays	
Section 3 — Singly Linked List Representation	Page 5
Node Structure, Memory Representation, Diagram	
Section 4 — Operations on Singly Linked List	Page 6–8
Insertion (Beginning, End, Middle), Deletion, Traversal	
Section 5 — Doubly Linked List	Page 9
Structure, Node Representation, Diagram	
Section 6 — Operations on Doubly Linked List	Page 10
Insertion, Deletion, Traversal Algorithms	
Section 7 — Circular Linked List	Page 11
Concept, Structure, Circular SLL & DLL	
Section 8 — Operations on Circular Linked List	Page 12
Section 9 — Comparison: Arrays vs Linked Lists	Page 13
Section 10 — Memory Representation	Page 14
Section 11 — Applications of Linked Lists	Page 15
Section 12 — Advantages & Disadvantages	Page 16
Section 13 — Questions & Answers (2-mark & 5-mark)	Page 17–19
Section 14 — Quick Revision Sheet	Page 20
Textbook References	Page 21

★ HOW TO USE THIS MATERIAL

This document covers the complete B.Tech 1st Year Data Structures Unit II syllabus. Each section includes definitions, algorithms in C-style pseudocode, diagrams, time complexity analysis, and exam-focused Q&A. Use the Quick Revision Sheet (Section 14) before exams.

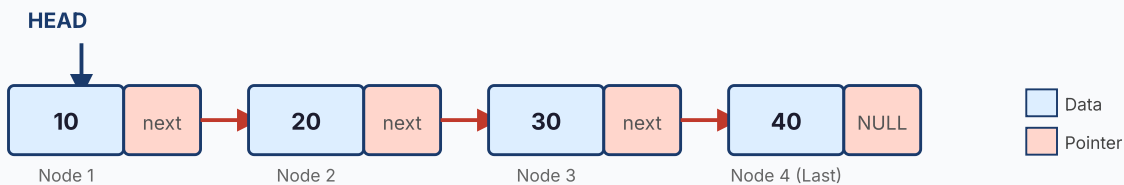
1 Introduction to Linked Lists

1.1 Definition

A **Linked List** is a linear data structure in which elements (called **nodes**) are stored at non-contiguous memory locations and are linked together using **pointers**. Unlike arrays, the elements of a linked list are not stored at adjacent memory addresses. Each node contains the data and the address (pointer) of the next node in the sequence.

Formally: A linked list is a collection of nodes where each node has two parts — a **data field** storing the actual value, and a **link field** (pointer) storing the address of the next node.

FIG 1.1 — BASIC LINKED LIST STRUCTURE



1.2 Terminology

Term	Meaning
Node	Basic unit of a linked list containing data and a pointer
Head	Pointer to the first node of the list
Tail	Last node whose next pointer is NULL
NULL	End marker indicating no further node exists
Link / Pointer	Address field storing reference to next node
Data Field	Stores the actual value of the node

1.3 Types of Linked Lists

- **Singly Linked List (SLL)** — Each node points to the next node only
- **Doubly Linked List (DLL)** — Each node points to both next and previous nodes
- **Circular Linked List (CLL)** — Last node points back to the first node
- **Circular Doubly Linked List** — Combines DLL and circular properties

KEY FACTS

Node structure in C: `struct Node { int data; struct Node *next; };`

Memory per node = `sizeof(data) + sizeof(pointer)`

Pointer size on 32-bit system = 4 bytes; on 64-bit = 8 bytes

2 Need for Linked Lists

2.1 Limitations of Arrays

Arrays are the most basic data structure but suffer from several significant limitations that necessitate the use of linked lists:

#	Limitation of Arrays	How Linked List Solves It
1	Fixed size — must declare size at compile time	Dynamic size — grows/shrinks at runtime
2	Insertion/Deletion requires shifting elements — $O(n)$	Insertion/Deletion at beginning — $O(1)$
3	Memory wastage if array is not fully used	Allocates only what is needed
4	Contiguous memory block required (may not be available)	Non-contiguous memory — flexible allocation
5	Cannot efficiently insert between elements	Efficient middle insertion with pointer change

2.2 Dynamic Memory Allocation

Linked lists use **dynamic memory allocation** — memory is requested from the heap at runtime as nodes are created, and released when nodes are deleted. This is managed using:

```
C — DYNAMIC NODE CREATION & DELETION
```

```
struct Node {  
    int data;  
    struct Node *next;  
};  
  
/* Allocate new node */  
struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));  
newNode->data = 10;  
newNode->next = NULL;  
  
/* Free node */  
free(newNode);
```

2.3 When to Use Linked Lists

- ✓ When the number of elements is **not known in advance**
- ✓ When **frequent insertions and deletions** are required

- ✓ When implementing **stacks, queues, graphs, trees**
- ✓ When **random access is not critical**

✦ **EXAM NOTE**

The key difference between arrays and linked lists is that arrays provide

RANDOM ACCESS $O(1)$

while linked lists require

SEQUENTIAL ACCESS $O(N)$

. This trade-off is central to many exam questions.

3.1 Definition

A **Singly Linked List (SLL)** is a type of linked list where each node contains a data field and a single pointer to the next node. Traversal is possible only in the **forward direction**.

3.2 Node Structure

FIG 3.1 — NODE STRUCTURE OF SINGLY LINKED LIST

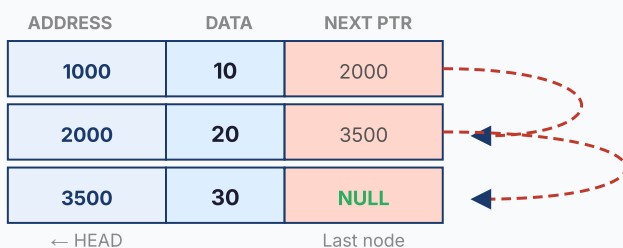


C STRUCTURE DEFINITION — SINGLY LINKED LIST NODE

```
struct Node {  
    int data; /* Data field */  
    struct Node *next; /* Pointer to next node */  
};  
  
struct Node *head = NULL; /* Initially empty list */
```

3.3 Memory Representation

FIG 3.2 — MEMORY REPRESENTATION OF SINGLY LINKED LIST



Key Points:

- Nodes stored at non-contiguous addresses
- Each node's NEXT holds address of next node
- Last node's NEXT = NULL (end marker)
- HEAD pointer tracks start of the list

SLL PROPERTIES

Size of Node = `sizeof(data) + sizeof(pointer)`

Space Complexity = $O(n)$

Traversal direction: Forward only

Access: Sequential $O(n)$

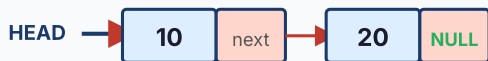
4 Operations on Singly Linked List

4.1 Insertion at the Beginning

A new node is inserted before the current head node. The new node becomes the new head.

FIG 4.1 — INSERTION AT BEGINNING (INSERTING 5 BEFORE EXISTING LIST)

BEFORE:



AFTER (new node 5 inserted at beginning):



Steps:

- ① Create new node with data=5
- ② Set newNode→next = HEAD
- ③ Set HEAD = newNode

ALGORITHM – INSERT AT BEGINNING

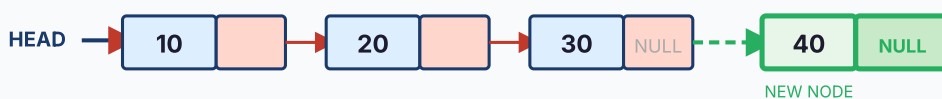
```
insertAtBeginning(head, data):  
    newNode = malloc(sizeof(Node))  
    newNode.data = data  
    newNode.next = head // new node points to old head  
    head = newNode      // head updated to new node  
    return head
```

⌚ TIME COMPLEXITY

Insertion at Beginning = $O(1)$ | Insertion at End = $O(n)$ | Insertion at Middle = $O(n)$

4.2 Insertion at the End

FIG 4.2 — INSERTION AT END (APPENDING NODE WITH DATA=40)



- ① Traverse to last node (whose next = NULL)
- ② Create new node; set newNode→next = NULL
- ③ Set lastNode→next = newNode

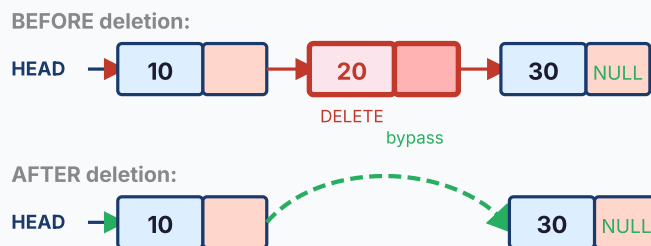
```

insertAtEnd(head, data):
    newNode = malloc(sizeof(Node))
    newNode.data = data; newNode.next = NULL
    if head == NULL: return newNode
    temp = head
    while temp.next != NULL:
        temp = temp.next // traverse to last
    temp.next = newNode
    return head

```

4.3 Deletion of a Node

FIG 4.3 — DELETION OF NODE WITH DATA=20



```

deleteNode(head, key):
    if head == NULL: return NULL
    if head.data == key:
        temp = head; head = head.next; free(temp); return head
    prev = head; curr = head.next
    while curr != NULL:
        if curr.data == key:
            prev.next = curr.next // bypass the node
            free(curr); return head
        prev = curr; curr = curr.next
    return head // key not found

```

4.4 Traversal

ALGORITHM – TRAVERSE / DISPLAY SINGLY LINKED LIST

```
traverseSLL(head):  
    temp = head  
    while temp != NULL:  
        print(temp.data) // visit each node  
        temp = temp.next // move to next node
```

SLL TIME COMPLEXITY SUMMARY

Insert at Beginning: $O(1)$

Insert at End: $O(n)$

Delete by Value: $O(n)$

Search: $O(n)$

Traversal: $O(n)$

Space: $O(n)$

5 Doubly Linked List

5.1 Definition

A **Doubly Linked List (DLL)** is a linked list where each node contains three fields: a **data field**, a pointer to the **next node (next)**, and a pointer to the **previous node (prev)**. Traversal is possible in **both forward and backward** directions.

5.2 Node Structure

FIG 5.1 — DOUBLY LINKED LIST NODE STRUCTURE

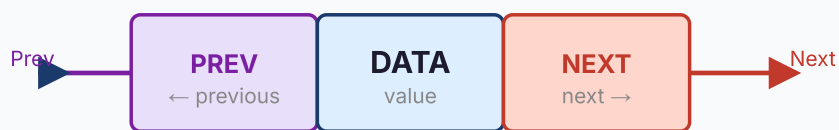
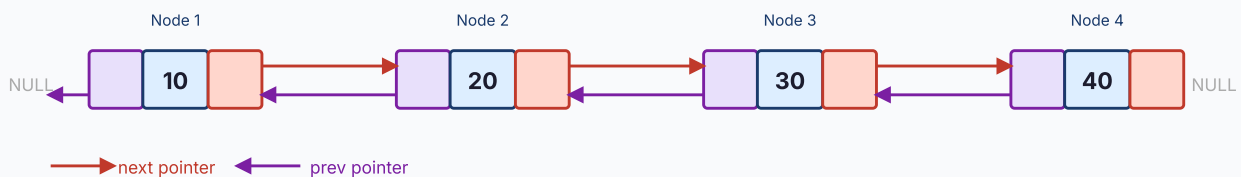


FIG 5.2 — DOUBLY LINKED LIST STRUCTURE (4 NODES)



C STRUCTURE — DOUBLY LINKED LIST NODE

```
struct DNode {  
    int data;  
    struct DNode *prev; /* pointer to previous node */  
    struct DNode *next; /* pointer to next node */  
};
```

6 Operations on Doubly Linked List

6.1 Insertion at Beginning

ALGORITHM – DLL INSERT AT BEGINNING

```
DLL_insertBeg(head, data):  
    newNode = malloc(sizeof(DNode))  
    newNode.data = data  
    newNode.prev = NULL  
    newNode.next = head  
    if head != NULL: head.prev = newNode  
    head = newNode  
    return head
```

6.2 Insertion at End

ALGORITHM – DLL INSERT AT END

```
DLL_insertEnd(head, data):  
    newNode = malloc(sizeof(DNode))  
    newNode.data = data; newNode.next = NULL  
    if head == NULL: newNode.prev = NULL; return newNode  
    temp = head  
    while temp.next != NULL: temp = temp.next  
    temp.next = newNode; newNode.prev = temp  
    return head
```

6.3 Deletion of a Node

ALGORITHM – DLL DELETE BY VALUE

```
DLL_delete(head, key):  
    temp = head  
    while temp != NULL and temp.data != key:  
        temp = temp.next  
    if temp == NULL: return head // not found  
    if temp.prev != NULL: temp.prev.next = temp.next  
    else: head = temp.next // deleting head  
    if temp.next != NULL: temp.next.prev = temp.prev  
    free(temp)  
    return head
```

6.4 Traversal (Forward and Backward)

ALGORITHM – DLL TRAVERSAL

```
/* Forward Traversal */
DLL_forwardTraversal(head):
    temp = head
    while temp != NULL:
        print(temp.data); temp = temp.next

/* Backward Traversal */
DLL_backwardTraversal(head):
    temp = head
    while temp.next != NULL: temp = temp.next
    while temp != NULL:
        print(temp.data); temp = temp.prev
```

🕒 DLL TIME COMPLEXITY

Insert at Beginning: $O(1)$ Insert at End: $O(n)$ Delete: $O(n)$
Forward Traversal: $O(n)$ Backward Traversal: $O(n)$
Extra space per node: 1 pointer (prev)

Operation	Singly LL	Doubly LL
Backward traversal	✗ Not possible	✓ $O(n)$
Delete given pointer	✗ Need prev node	✓ $O(1)$
Extra memory per node	1 pointer	2 pointers
Insert at beginning	$O(1)$	$O(1)$
Implementation	Simpler	More complex

Table 6.1 — Singly LL vs Doubly LL Comparison

7.1 Definition and Concept

A **Circular Linked List (CLL)** is a linked list in which the last node's next pointer does not point to NULL, but instead points back to the **first node (head)**, forming a circle. There is no NULL pointer at the end.

There are two types: **Circular Singly Linked List** (each node has one pointer pointing forward) and **Circular Doubly Linked List** (each node has two pointers, forming a bidirectional circle).

FIG 7.1 — CIRCULAR SINGLY LINKED LIST STRUCTURE

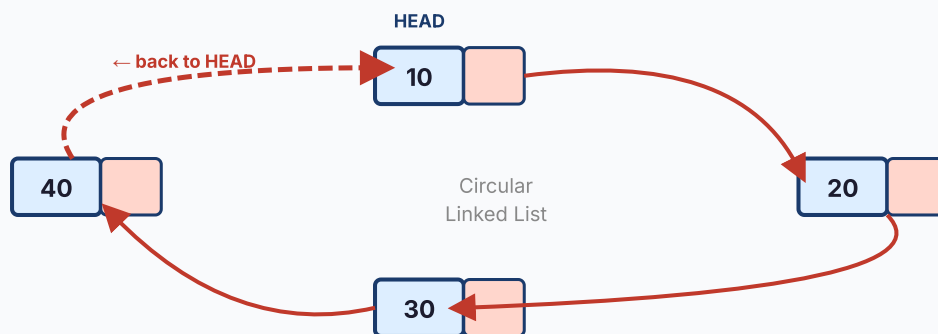
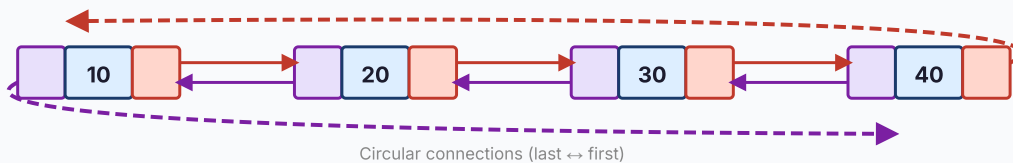


FIG 7.2 — CIRCULAR DOUBLY LINKED LIST STRUCTURE



💡 KEY PROPERTY

In a circular linked list, you can reach any node from any other node. There is no definite beginning or end — this makes it ideal for applications like round-robin scheduling.

🔗 IDENTIFYING CIRCULAR LINKED LIST

Condition: `last_node → next == head` (NOT NULL)

Traversal termination: stop when `temp.next == head` (not NULL)

8 Operations on Circular Linked List

8.1 Insertion at Beginning (CLL)

ALGORITHM – CLL INSERT AT BEGINNING

```
CLL_insertBeg(head, data):
    newNode = malloc(sizeof(Node))
    newNode.data = data
    if head == NULL:
        newNode.next = newNode // points to itself
        return newNode
    temp = head
    while temp.next != head: temp = temp.next // go to last
    newNode.next = head
    temp.next = newNode // last node points to new node
    return newNode // new node is new head
```

8.2 Insertion at End (CLL)

ALGORITHM – CLL INSERT AT END

```
CLL_insertEnd(head, data):
    newNode = malloc(sizeof(Node))
    newNode.data = data
    if head == NULL:
        newNode.next = newNode; return newNode
    temp = head
    while temp.next != head: temp = temp.next
    temp.next = newNode; newNode.next = head
    return head
```

8.3 Traversal of CLL

ALGORITHM – CLL TRAVERSAL

```
CLL_traverse(head):
    if head == NULL: return
    temp = head
    do:
        print(temp.data)
        temp = temp.next
    while temp != head // stop when we reach head again
```


8.4 Deletion in CLL

ALGORITHM – CLL DELETE BY KEY

```
CLL_delete(head, key):
    if head == NULL: return NULL
    curr = head; prev = NULL
    while curr.data != key:
        prev = curr; curr = curr.next
    if curr == head: return head // not found
    if curr == head: // deleting head
        prev = head
        while prev.next != head: prev = prev.next
        head = head.next; prev.next = head
    else: prev.next = curr.next
    free(curr); return head
```

🕒 CLL TIME COMPLEXITY

Insert (Beginning/End): $O(n)$ – must find last node

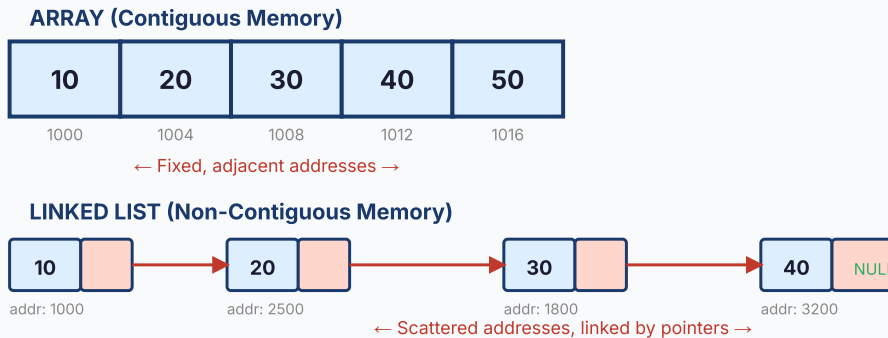
Traversal: $O(n)$

Delete: $O(n)$

Feature	SLL	DLL	CLL
NULL pointer at end	Yes	Yes	No (points to head)
Pointers per node	1	2	1
Backward traversal	No	Yes	No (unless CDLL)
Traversal termination	next == NULL	next == NULL	next == head
Applications	Stack, Queue	Browser history	Round-robin scheduling

Table 8.1 — Comparison: SLL vs DLL vs CLL

FIG 9.1 — ARRAY VS LINKED LIST MEMORY LAYOUT



Parameter	Array	Linked List
Memory Allocation	Static (compile time)	Dynamic (runtime)
Memory Layout	Contiguous	Non-contiguous
Size	Fixed	Dynamic (grows/shrinks)
Random Access	$O(1)$ — <code>array[i]</code>	$O(n)$ — must traverse
Insertion at Beginning	$O(n)$ — shift required	$O(1)$
Insertion at End	$O(1)$ if space available	$O(n)$
Deletion	$O(n)$ — shift required	$O(n)$ — search + pointer
Memory waste	Possible (unused slots)	Overhead of pointers
Cache Performance	Better (locality)	Poor (scattered)
Reverse traversal	Easy (index-based)	Needs DLL
Implementation	Simple	Complex
Use case	Known, fixed data size	Unknown, dynamic data

Table 9.1 — Comprehensive Comparison: Arrays vs Linked Lists

DECISION RULE

Use ARRAY when: random access needed, size is known, memory is limited.

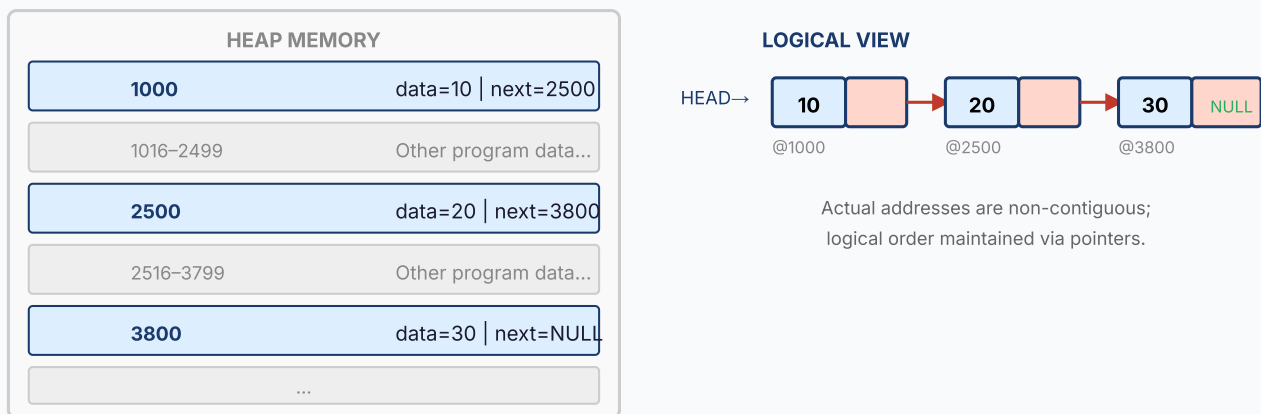
Use LINKED LIST when: frequent insertions/deletions, unknown size, implementing abstract data types.

10 Memory Representation of Linked Lists

10.1 Heap Memory and Pointers

Linked list nodes are allocated from the **heap memory** using `malloc()` in C. Each node occupies a distinct block of heap memory. The OS returns a pointer to that block. These blocks can be anywhere in memory — not necessarily adjacent.

FIG 10.1 — HEAP MEMORY WITH LINKED LIST NODES



10.2 Stack vs Heap

Property	Stack Memory	Heap Memory
Allocation	Automatic (function call)	Manual (malloc/free)
Size	Limited (few MB)	Large (available RAM)
Lifetime	Until function returns	Until explicitly freed
Used by	Local variables, arrays	Linked list nodes
Fragmentation	No	Possible

⚠ MEMORY LEAK WARNING

Always `free()` allocated nodes when deleting from a linked list. Failing to do so causes memory leaks — the program consumes more and more memory over time until it crashes.

11 Applications of Linked Lists

11.1 Implementation of Other Data Structures

Data Structure	LL Type Used	How LL Helps
Stack	SLL	Push = insert at head ($O(1)$), Pop = delete head ($O(1)$)
Queue	SLL	Enqueue at tail, Dequeue at head — both $O(1)$ with tail pointer
Deque	DLL	Insert/delete at both ends efficiently
Hash Table	SLL	Chaining for collision resolution
Adjacency List (Graph)	SLL	Store neighbors of each vertex
Tree	DLL	Parent/child pointers

11.2 Real-World Applications

- 1 **Browser History:** DLL stores visited pages — forward (next) and back (prev) navigation
- 2 **Music Player Playlist:** CLL allows endless playback — last song wraps back to first
- 3 **OS Process Scheduling:** CLL in round-robin CPU scheduling — each process gets a time slice
- 4 **Undo/Redo in Text Editors:** DLL stores states — undo moves backward, redo moves forward
- 5 **Polynomial Representation:** Each node stores coefficient and exponent; linked for efficient operations
- 6 **Memory Management (Free List):** OS maintains free memory blocks as a linked list
- 7 **Symbol Table in Compilers:** Hash table with chaining uses linked lists
- 8 **Sparse Matrix:** Only non-zero elements stored as nodes — saves memory
- 9 **Fibonacci Heap:** Advanced data structure using DLL for priority queues
- 10 **Image Viewer:** CLL for cycling through photos in a gallery

★ EXAM TIP

For application questions, always mention: (1) which type of LL is used, (2) why that type is suitable (which property makes it efficient), and (3) give a real-world example.

12 Advantages & Disadvantages

12.1 Advantages of Linked Lists

#	Advantage	Explanation
1	Dynamic Size	Size can grow or shrink at runtime — no fixed allocation needed
2	Efficient Insertion/Deletion	Adding or removing at beginning takes $O(1)$ — no shifting needed
3	No Memory Wastage	Allocates only as much as needed — no unused space
4	Flexible	Easily used to implement stacks, queues, and other ADTs
5	No Contiguous Block	Can use fragmented heap memory — doesn't require a large continuous block
6	Easy Merge/Split	Two lists can be merged in $O(1)$ by connecting tail to head

12.2 Disadvantages of Linked Lists

#	Disadvantage	Explanation
1	No Random Access	Cannot access element by index — must traverse from head $O(n)$
2	Extra Memory	Each node needs extra memory for pointer(s) — overhead
3	Poor Cache Performance	Non-contiguous memory = more cache misses than arrays
4	Traversal Overhead	Must traverse from head for every access — no binary search
5	Risk of Memory Leak	Improper <code>free()</code> causes memory leaks in C
6	Reverse Traversal	SLL cannot be traversed backwards — need DLL
7	More Complex Code	Pointer manipulation is error-prone compared to arrays

QUICK COMPARISON SUMMARY

Best use of LL: Frequent insert/delete, dynamic data

Best use of Array: Random access, cache-sensitive code

SLL: Simple, one direction

DLL: Two directions, extra memory

CLL: Circular apps, no NULL end

13 Questions & Answers

2-Mark Questions

Q1. What is a Linked List?

2 Marks

A linked list is a linear data structure where elements called nodes are stored at non-contiguous memory locations and are connected using pointers. Each node has two parts: a data field holding the actual value, and a link (pointer) field holding the address of the next node. The first node is accessed via a pointer called HEAD, and the last node's pointer is set to NULL to indicate the end of the list. Unlike arrays, linked lists do not require contiguous memory and can grow or shrink dynamically.

Q2. What is the difference between a Singly and Doubly Linked List?

2 Marks

A Singly Linked List (SLL) has nodes with one pointer (next) pointing to the next node, allowing traversal only in the forward direction. A Doubly Linked List (DLL) has nodes with two pointers — next (to the next node) and prev (to the previous node) — enabling bidirectional traversal. DLL nodes require more memory (one extra pointer per node) but support operations like backward traversal and deletion given only the node pointer ($O(1)$) that SLL cannot perform efficiently.

Q3. What is a Circular Linked List?

2 Marks

A Circular Linked List is a type of linked list in which the last node's next pointer points back to the first (head) node instead of pointing to NULL. This forms a circular chain with no definite beginning or end. It can be singly circular (one pointer per node) or doubly circular (two pointers). The traversal continues until we revisit the head node. Circular linked lists are useful for round-robin scheduling, circular buffers, and applications that require repeated cycling through elements.

Q4. What is the time complexity of insertion at the beginning of a linked list?

2 Marks

The time complexity of inserting a node at the beginning of a singly linked list is **$O(1)$** — constant time. This is because the operation only involves creating a new node, setting its next pointer to the current head, and updating the head pointer to the new node. No traversal is needed. This is a major advantage over arrays, where inserting at the beginning requires shifting all n elements to the right, taking $O(n)$ time.

Q5. What are the advantages of linked list over array?

2 Marks

Linked lists have several advantages over arrays: (1) Dynamic size — size can grow or shrink at runtime without pre-allocation. (2) Efficient insertion and deletion at the beginning takes $O(1)$ compared to $O(n)$ for arrays. (3) No memory wastage — memory is allocated only when needed. (4) No large contiguous memory block required — scattered heap blocks can be used. (5) Easy to implement stacks, queues, and other dynamic data structures. However, arrays are preferred when random access is needed since linked lists require $O(n)$ for element access.

5-Mark Questions

Q6. Explain Singly Linked List with a neat diagram.

5 Marks

Definition: A Singly Linked List (SLL) is a linear data structure where each node contains a data field and a pointer (next) to the next node. The traversal is possible only in the forward direction.

Structure: Each node is defined as: `struct Node { int data; struct Node *next; };`

Working: The HEAD pointer stores the address of the first node. Each node's next field holds the address of the subsequent node. The last node's next is NULL.

Operations:

- *Traversal:* Start at HEAD, visit each node following next pointers until NULL — $O(n)$
- *Insertion at Beginning:* Create node → set next = HEAD → update HEAD = newNode — $O(1)$
- *Insertion at End:* Traverse to last node → set last.next = newNode — $O(n)$
- *Deletion:* Find predecessor of target → update predecessor.next = target.next → free target — $O(n)$

Time Complexity: Insert at beginning $O(1)$, all others $O(n)$. Space complexity $O(n)$.

Example: HEAD → [10|●] → [20|●] → [30|NULL]

Applications: Used in implementing stacks, queues, symbol tables in compilers, and adjacency lists in graphs.

Insertion Operations:

1. *At Beginning:* Create a new node. Set $\text{newNode} \rightarrow \text{next} = \text{HEAD}$. Set $\text{HEAD} = \text{newNode}$. Time: $O(1)$. The new node becomes the first node without traversal.
2. *At End:* Create new node. If list empty, return newNode as HEAD . Otherwise traverse using a temp pointer until $\text{temp} \rightarrow \text{next} == \text{NULL}$. Set $\text{temp} \rightarrow \text{next} = \text{newNode}$. Time: $O(n)$ due to traversal.
3. *At Position:* Traverse to $(\text{position}-1)$ th node. Set $\text{newNode} \rightarrow \text{next} = \text{node} \rightarrow \text{next}$. Set $\text{node} \rightarrow \text{next} = \text{newNode}$. Time: $O(n)$.

Deletion Operations:

1. *Delete Head:* Set $\text{temp} = \text{HEAD}$. Update $\text{HEAD} = \text{HEAD} \rightarrow \text{next}$. Free(temp). Time: $O(1)$.
2. *Delete by Value:* Traverse with prev and curr pointers. When $\text{curr} \rightarrow \text{data} == \text{key}$, set $\text{prev} \rightarrow \text{next} = \text{curr} \rightarrow \text{next}$. Free curr. Time: $O(n)$.
3. *Delete Last:* Traverse to second-last node. Set $\text{second-last} \rightarrow \text{next} = \text{NULL}$. Free last node. Time: $O(n)$.

Key Rule: Always save the node to be freed in a temp pointer before changing links, then call $\text{free}(\text{temp})$.

Arrays store elements in contiguous memory locations. Size must be declared at compile time. Elements can be accessed in $O(1)$ time using index. Insertion/deletion in the middle requires shifting all subsequent elements — $O(n)$.

Linked Lists store nodes at non-contiguous locations connected by pointers. Size is dynamic. Access requires traversal from HEAD — $O(n)$. Insertion/deletion at beginning is $O(1)$ — only pointer adjustment needed.

Key Differences:

- Memory: Array = contiguous block; LL = scattered nodes via pointers
- Access: Array = $O(1)$ random access; LL = $O(n)$ sequential only
- Size: Array = fixed; LL = dynamic
- Insert at start: Array = $O(n)$ shifting; LL = $O(1)$
- Memory overhead: Array = none; LL = pointer per node
- Cache performance: Array = better (locality); LL = poor (scattered)

When to use Array: Fixed size, frequent random access, numerical computation.

When to use LL: Unknown size, frequent insertions/deletions, implementing stacks/queues.

Definition: A Circular Linked List is a linked list where the last node's next pointer points back to the first (head) node, forming a circle. There is no NULL pointer at the end.

Types: Circular Singly Linked List (CSLL) — one pointer per node; Circular Doubly Linked List (CDLL) — two pointers per node with circular connections.

Structure: HEAD $\rightarrow$ [10|•] $\rightarrow$ [20|•] $\rightarrow$ [30|•] $\rightarrow$ (back to HEAD)

Operations:

- *Traversal:* Start at HEAD, use a do-while loop, continue until $\text{temp} \rightarrow \text{next} == \text{HEAD}$. Cannot use NULL check.
- *Insert at Beginning:* Create node $\rightarrow$ find last node $\rightarrow$ set $\text{newNode} \rightarrow \text{next} = \text{HEAD} \rightarrow \text{last} \rightarrow \text{next} = \text{newNode} \rightarrow \text{HEAD} = \text{newNode}$. $O(n)$
- *Insert at End:* Traverse to last $\rightarrow \text{last} \rightarrow \text{next} = \text{newNode} \rightarrow \text{newNode} \rightarrow \text{next} = \text{HEAD}$. $O(n)$
- *Delete:* Find node to delete; update its predecessor's next; if deleting head, update head and $\text{last} \rightarrow \text{next}$. $O(n)$

Applications: Round-robin CPU scheduling, music playlist cycling, circular buffer (ring buffer), multiplayer game turns, token ring network protocol.

Key property: Every node is reachable from every other node — no definite beginning or end.

 Key Definitions

- **Node:** Data + Pointer(s)
- **HEAD:** Pointer to first node
- **SLL:** One pointer (next)
- **DLL:** Two pointers (prev + next)
- **CLL:** last → next = HEAD
- **NULL:** End marker of SLL/DLL
- **Dynamic alloc:** malloc() / free()

 Time Complexities

- Insert at beginning: **$O(1)$**
- Insert at end: **$O(n)$**
- Insert at position: **$O(n)$**
- Delete by value: **$O(n)$**
- Delete head: **$O(1)$**
- Traversal: **$O(n)$**
- Search: **$O(n)$**
- Space: **$O(n)$**

 SLL vs DLL vs CLL

- SLL: 1 ptr/node, forward only
- DLL: 2 ptrs/node, bidirectional
- CLL: last → next = HEAD
- CLL traversal ends when next == HEAD
- DLL delete given ptr: $O(1)$
- SLL delete given ptr: $O(n)$

 Array vs Linked List

- Array access: $O(1)$ | LL: $O(n)$
- Array insert at start: $O(n)$ | LL: $O(1)$
- Array: contiguous | LL: scattered
- Array: fixed size | LL: dynamic
- Array: cache friendly | LL: not

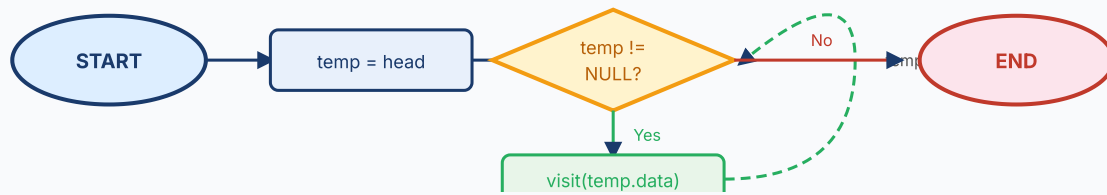
 Applications

- SLL → Stack, Queue, Hash Chaining
- DLL → Browser history, Undo/Redo
- CLL → Round-robin, Music playlist
- LL → Adjacency list, Sparse matrix
- LL → OS free memory list

 Important Notes

- Always free() deleted nodes
- Check if HEAD == NULL before delete
- CLL: use do-while for traversal
- DLL saves the prev pointer update
- Tail pointer → $O(1)$ end insert in SLL
- LL has no random access (binary search impossible)

QUICK REFERENCE — TRAVERSAL FLOWCHART



🎯 LAST-MINUTE FORMULA CARD

n = number of nodes

Node size = data + pointer(s)

SLL: n pointers | DLL: $2n$ pointers

Best case search: $O(1)$ (first node)

Worst case: $O(n)$ (last node or not found)



Textbook References

Primary Textbooks

- T1 Mark Allen Weiss** — *Data Structures and Algorithm Analysis in C*, 2nd Edition. Pearson Education. Chapters 3 (Lists, Stacks, Queues) covers SLL, DLL, and their implementations thoroughly with C code examples.
- T2 Ellis Horowitz, Sartaj Sahni** — *Fundamentals of Data Structures in C*, 2nd Edition. Universities Press. Chapter 4 covers linked list representation, operations, circular lists, and doubly linked lists with detailed algorithms.

Reference Books

- R1 Kurt Mehlhorn & Peter Sanders** — *Algorithms and Data Structures: The Basic Toolbox*. Springer. Provides theoretical analysis of linked list operations with mathematical proofs.
- R2 Aho, Ullman & Hopcroft** — *Data Structures and Algorithms*. Addison-Wesley. Covers linked lists in the context of abstract data types with formal definitions and correctness arguments.
- R3 Brad Miller & David Ranum** — *Problem Solving with Algorithms and Data Structures using Python*. Franklin, Beedle & Associates. Contains conceptual linked list explanations suitable for building intuition.
- R4 Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein** — *Introduction to Algorithms (CLRS)*, 3rd Edition. MIT Press. Chapter 10 covers elementary data structures including linked lists with rigorous pseudo-code.
- R5 Robert Sedgewick** — *Algorithms in C, Parts 1–4*, 3rd Edition. Addison-Wesley. Detailed C implementations of linked lists and their applications in real programs.

★ STUDY RECOMMENDATION

For exam preparation: Start with

HOROWITZ & SAHNI (T2)

for clear definitions and algorithms. Use

MARK ALLEN WEISS (T1)

for complexity analysis. Refer to

CLRS (R4)

for conceptual depth. Practice all algorithms manually with sample inputs before the exam.

